

MVGR COLLEGE OF ENGINEERING (AUTONOMOUS)
VIZIANAGARAM
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**LAB RECORD**

Course	Mean Stack Web Development
Academic Year	2026 - 2027
Student Name	K Vijay Kumar Dev
Registration Number	23331A05A2
Section	B
Lab In-Charge	Mr. R. Ravi Kanth

Signature of Lab In-Charge

Head of the Department - CSE

INDEX SHEET

S. No.	Week	Experiment / Activity Title	Date	Page No.	Grade	Signature of Lab In-Charge
1	1	jQuery Selectors and Content Manipulation		4		
2	2	Changing Webpage Style Dynamically using jQuery		6		
3	3	jQuery DOM Manipulation and Traversing		11		
4	4	Event Handling using jQuery		17		
5	5	Student Registration Page and Student Information Table using Bootstrap		22		
6	6	Data Binding and Basic AngularJS Directives		27		
7	7	AngularJS Built-in Filters		31		
8	8	Student Registration Form using AngularJS		34		
9	9	AngularJS Service and Routing Application		39		
10	10	Node.js Built-in Modules - fs, http, path, os and events		50		
11	11	User-Defined Modules for Arithmetic Operations and Student Details		55		
12	12	Node.js Event Emitter and Streams		59		
13	13	Student Information Web Server using Node.js		63		

14	14	Student Application using Express.js		66		
15	15	Student Database Application using MongoDB and Express.js		74		

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
1	1		2

1. TITLE / AIM

jQuery Selectors and Content Manipulation

To design an HTML page that displays student details such as name, roll number, branch and marks, and to manipulate the text, HTML content and attributes of the selected elements using jQuery selectors.

2. OBJECTIVES

- To create a static HTML page containing student details.
- To select elements using id, class, element and attribute selectors in jQuery.
- To modify element content using text(), html() and val() methods.
- To read and modify element attributes using attr() and removeAttr().
- To display the modified student details dynamically on the webpage.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Operating System: Windows 10 / Linux.
- Software: Any modern web browser (Google Chrome / Mozilla Firefox).
- Editor: Visual Studio Code.
- Library: jQuery 3.7.1 (loaded through a CDN).
- Languages: HTML5, CSS3, JavaScript (jQuery).

4. THEORY / CONCEPT jQuery is a lightweight, cross-browser JavaScript library that simplifies HTML document traversal, event handling and DOM manipulation. Every jQuery statement follows the basic syntax \$(selector).action(), where the selector identifies the HTML element(s) and the action is the method applied to them.

A jQuery selector uses CSS-style syntax to locate elements in the DOM and returns a jQuery object that may contain one or more elements. The commonly used categories of selectors are listed below.

- Element selector: \$("p") selects every paragraph in the document.
- ID selector: \$("#sname") selects the single element whose id is sname.
- Class selector: \$(".marks") selects all elements carrying the class marks.
- Attribute selector: \$("input[type='text']") selects elements by attribute value.
- Grouping and hierarchy: \$("h1, h2"), \$("div p"), \$("ul > li").

Content manipulation is performed using three primary methods. The text() method sets or returns only the text content of an element and escapes any markup. The html() method sets or

returns the inner HTML, so markup passed to it is rendered. The `val()` method sets or returns the value of form input elements. When these methods are called without an argument they act as getters; when called with an argument they act as setters.

Attributes are handled using `attr(name)` to read an attribute, `attr(name, value)` to set one, `attr({...})` to set several at once, and `removeAttr(name)` to delete an attribute. The `$(document).ready()` function ensures that jQuery code executes only after the entire DOM has been loaded, which prevents manipulation of elements that do not yet exist.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create an HTML page with student details and an input field for marks.
- Step 2: Include jQuery and select elements using different selectors.
- Step 3: Add a button click event to modify the selected elements.
- Step 4: Modify content using `text()`, `html()`, and `val()` methods.
- Step 5: Add and remove attributes using `attr()` and `removeAttr()`.
- Step 6: Display the updated student details on the webpage.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Selectors</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

<h2 id="name">Student: Ravi</h2>
<p class="details">Roll No: 101</p> <p
class="details">Branch: CSE</p>
<input id="marks" value="85">
<button id="btn">Modify</button>
<script>
$(document).ready(function () {
```

```
$("#btn").click(function () {  
    $("#name").text("Student: Rahul");    // id selector + text()  
    $(".details").html("<b>Branch: CSE</b>");    // class selector + html()  
    $("input").val("90");    // element selector + val()  
    $("input[value]").attr("title", "Updated");    // attribute selector + attr()  
    $("#name").removeAttr("style");    // removeAttr() });  
});  
  
</script>  
</body>  
</html>
```

7. OBSERVATION / OUTPUT



Student: Ravi

Roll No: 101

Branch: CSE



Student: Rahul

Roll No: 101

Branch: CSE

8. RESULT / INFERENCE

The webpage displaying student details was created successfully and the elements were selected using id, class and element selectors. The text, inner HTML and attributes of the selected elements were modified dynamically using text(), html() and attr(), and the updated student details were displayed on the webpage. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOMES

The following competencies were acquired through this experiment:

- LO1: Ability to include the jQuery library in an HTML page and write code inside `$(document).ready()`.
- LO2: Ability to select DOM elements using element, id, class and attribute selectors.
- LO3: Ability to distinguish between `text()`, `html()` and `val()` and to modify element attributes using `attr()` and `removeAttr()`.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
2	2		6

1. TITLE / AIM

Changing Webpage Style Dynamically using jQuery

To create a webpage containing headings, paragraphs, buttons and a student table, and to change the background colour, font colour, font size and border of the selected elements using jQuery, along with the dynamic addition and removal of CSS classes.

2. OBJECTIVES

- To design a webpage with headings, paragraphs, buttons and a student table.
- To apply inline CSS properties to selected elements using the `css()` method.
- To change background colour, font colour, font size and border at run time.
- To add, remove and toggle CSS classes using `addClass()`, `removeClass()` and `toggleClass()`.
- To understand the difference between applying inline styles and applying pre-defined classes.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Library: jQuery 3.7.1 (CDN).
- Languages: HTML5, CSS3, JavaScript (jQuery).

4. THEORY / CONCEPT jQuery provides a set of methods that modify the presentation of an element without reloading the page. The `css()` method is the central method for this purpose. It can be used in three forms: `css("property")` returns the computed value of the property, `css("property", "value")` sets a single property, and `css({property1: value1, property2: value2})` sets several properties at once through an object literal.

Styling through the `css()` method produces inline styles, which have a higher specificity than styles defined in a stylesheet. This is convenient for a single change, but when a group of properties has to be applied repeatedly it is better to define a CSS class and attach it to the element. jQuery supports this through the following class methods.

- `addClass("name")` attaches one or more classes to the matched elements.
- `removeClass("name")` detaches the specified class; calling it without an argument removes all classes.
- `toggleClass("name")` attaches the class if absent and detaches it if present.
- `hasClass("name")` returns true when at least one matched element carries the class. The commonly used style properties in this experiment are background-color, color, font-size

and border. In jQuery they may be written either in hyphenated CSS form ("background-color") or in camel case (backgroundColor). Combining the css() method with selectors such as \$("tr:even") or \$(".fail") makes it possible to style alternate rows or conditional data without editing the markup.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create a webpage with headings, paragraphs, buttons and a student table.
- Step 2: Include jQuery and define CSS classes for styling.
- Step 3: Add click events to the buttons.
- Step 4: Change colours, font size and border using css().
- Step 5: Add, remove or toggle CSS classes using jQuery methods.
- Step 6: Display the changed styles dynamically on the webpage.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Style Manipulation</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <style>
    .highlight { background: lightblue; color: darkblue; }
    .border { border: 2px solid black; }
  </style>
</head>
<body>
<h2 id="title">Student Details</h2>
<p id="text">This is a student information page.</p>
<table border="1" id="table">
  <tr><th>Name</th><th>Marks</th></tr>
  <tr><td>Ravi</td><td>85</td></tr>
  <tr><td>Rahul</td><td>90</td></tr>
```

```
</table><br>
<button id="style">Change Style</button>
<button id="class">Toggle Class</button>
<script>
$(document).ready(function() {
    $("#style").click(function() {
        $("#title").css({"color":"red","font-size":"30px"});
        $("#text").css({"background":"yellow","color":"green"});
        $("#table").css("border","3px solid blue");
    });
    $("#class").click(function() {
        $("#text").toggleClass("highlight");
        $("#table").toggleClass("border");
    });
});
</script>
</body>
</html>
```

7. OBSERVATION / OUTPUT



Student Details

This is a student information page.

Name	Marks
Ravi	85
Rahul	90

Change Style Toggle Class

After clicking Toggle Class button



Student Details

This is a student information page.

Name	Marks
Ravi	85
Rahul	90

Change Style

Toggle Class

After clicking the button Change Style



Student Details

This is a student information page.

Name	Marks
Ravi	85
Rahul	90

Change Style

Toggle Class

8. RESULT / INFERENCE

The webpage containing a heading, status paragraph, buttons and a student table was created successfully. The colour, font size, background colour, padding and border of the selected elements were changed at run time using the jQuery `css()` method, and CSS classes were dynamically added and removed using `toggleClass()`. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOMES

The following competencies were acquired through this experiment:

- LO1: Ability to apply CSS properties at run time using the jQuery `css()` method with an object literal.
- LO2: Ability to manage presentation through CSS classes using `toggleClass()`.
- LO3: Understanding of the difference between direct styling and class-based styling in jQuery.
- LO4: Ability to handle button click events and dynamically modify the appearance of HTML elements

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
3	3		11

1. TITLE / AIM

jQuery DOM Manipulation and Traversing

To create a webpage containing a list of students and to perform addition, removal and modification of list items, and to traverse the DOM to obtain the parent, children, first, last, previous and next elements of a selected student using jQuery.

2. OBJECTIVES

- To display a list of students using an unordered list.
- To add a new student to the list using `append()` and `prepend()`.
- To remove a student from the list using `remove()`.
- To change the name of a selected student using `text()`.
- To display the parent and child elements of a selected student using `parent()` and `children()`.
- To display the first and the last student using `first()` and `last()`.
- To display the previous and next siblings of a selected student using `prev()` and `next()`.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Library: jQuery 3.7.1 (CDN).
- Languages: HTML5, CSS3, JavaScript (jQuery).

4. THEORY / CONCEPT

DOM manipulation in jQuery refers to the insertion, deletion and modification of nodes in the document object model after the page has been loaded. The insertion methods are `append()` and `prepend()`, which add content inside the matched element at the end and at the beginning respectively, and `after()` and `before()`, which add content outside the matched element as a sibling. The removal methods are `remove()`, which deletes the element together with its data and events, and `empty()`, which deletes only the contents of the element while retaining the element itself.

Traversing refers to moving through the DOM tree starting from a selected element. The methods used in this experiment are grouped as follows.

- Upward traversal: `parent()` returns the direct parent, `parents()` returns all ancestors and `closest()` returns the nearest matching ancestor.

- Downward traversal: `children()` returns the direct children and `find()` returns all matching descendants.
- Sideways traversal: `siblings()` returns all siblings, `prev()` returns the immediately preceding sibling and `next()` returns the immediately following sibling.
- Filtering: `first()`, `last()`, `eq(index)` and `filter(selector)` reduce a set of matched elements to a smaller set.

The elements of a list are addressed collectively by `$("li")`; a single element is addressed by an index with `eq(index)`, or by the element that raised an event using the `$(this)` keyword. Since the nodes are live objects in the DOM, any structural change performed by one method is immediately visible to the subsequent traversal methods.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create a webpage with a list of student names.
- Step 2: Use jQuery to add, remove and modify list items.
- Step 3: Select a student and traverse its parent, children and sibling elements.
- Step 4: Use `first()` and `last()` to find the first and last students.
- Step 5: Display the traversed elements and updated list on the webpage.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery DOM Manipulation</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

<h2>Student List</h2>

<ul id="students">
  <li>Ravi</li>
  <li>Rahul</li>
  <li>Sita</li>
  <li>Anu</li>
</ul>

<button id="add">Add</button>
<button id="remove">Remove</button>
<button id="modify">Modify</button>
<button id="traverse">Traverse</button>
```

<p id="result"></p>

<script>

\$(function () {

\$("#add").click(function () {

\$("#students").append("Arun");

\$("#students").prepend("Priya");

});

\$("#remove").click(function () {

\$("#students li").first().remove();

});

\$("#modify").click(function () {

\$("#students li").first().text("Updated Student");

});

\$("#traverse").click(function () { let

s = \$("#students li").first();

\$("#result").html(

"Parent: " + s.parent().prop("tagName") +

"
Children: " + s.parent().children().length +

"
First: " + \$("#students li").first().text() +

"
Last: " + \$("#students li").last().text() +

"
Previous: " + (s.prev().text() || "None") +

"
Next: " + (s.next().text() || "None")

);

});

});

</script>

</body>

</html>

7. OBSERVATION / OUTPUT**Student List**

- Ravi
- Rahul
- Sita
- Anu

**Student List**

- Ravi
- Rahul Kumar
- Sita
- Anu

After clicking Add button

**Student List**

- Priya
- Ravi
- Rahul
- Sita
- Anu
- Arun

After clicking Remove Button



Student List

- Ravi
- Rahul
- Sita
- Anu
- Arun

After clicking Modify Last button



Student List

- Updated Student
- Rahul
- Sita
- Anu
- Arun

After clicking Modify Target Name button



Student List

- Updated Student
- Rahul
- Sita
- Anu
- Arun

After clicking Traverse button



Student List

- Updated Student
- Rahul
- Sita
- Anu
- Arun

Parent: UL
 Children: 5
 First: Updated Student
 Last: Arun
 Previous: None
 Next: Rahul

8. RESULT / INFERENCE

The webpage containing the student roster was created successfully. New students were added using `append()` and `prepend()`, students were removed using `remove()`, and the selected student's name was modified using `text()`. The parent, child, first, last, previous sibling and next sibling elements were successfully identified and displayed using the respective jQuery traversal methods. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOMES

The following competencies were acquired through this experiment:

- LO1: Ability to add and remove HTML elements dynamically using `append()`, `prepend()` and `remove()`.
- LO2: Ability to modify the text content of selected elements using `text()`.
- LO3: Ability to traverse the DOM using `parent()`, `children()`, `first()`, `last()`, `prev()` and `next()`.
- LO4: Understanding of the relationship between parent, child and sibling elements in the DOM.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
4	4		17

1. TITLE / AIM

Event Handling using jQuery

To create a webpage containing buttons, text boxes and student records, and to implement click, mouse and keyboard event handling along with the hiding and showing of student details using jQuery.

2. OBJECTIVES

- To change the background colour of an element when a button is clicked.
- To display a message when the mouse enters and leaves an element.
- To display the typed text as and when a key is pressed in a text box.
- To hide and show student details using button events.
- To understand the event object, event binding through on() and the use of \$(this).

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Library: jQuery 3.7.1 (CDN).
- Languages: HTML5, CSS3, JavaScript (jQuery).

4. THEORY / CONCEPT

An event is an action generated by the user or by the browser, such as a mouse click, a key press or the loading of a page. jQuery provides a uniform, cross-browser interface for attaching handler functions to such events. A handler may be attached through a shorthand method such as click(), or through the general method on("event", handler), which is preferred because it supports multiple events, delegated binding for elements created later and namespacing.

The events used in this experiment belong to three categories.

- Mouse events: click, dblclick, mouseenter, mouseleave, hover, mousedown and mouseup.
- Keyboard events: keydown, keypress and keyup. The keyup event is used for reading the current content of a text box because the typed character has already been placed in the box by that time.
- Form and display events: focus, blur, change and submit.

Every handler receives an event object that carries information such as event.type, event.target, event.which (the key code) and event.pageX/pageY. Inside the handler, the keyword \$(this) refers to the element on which the event occurred, which allows one handler to serve many elements. The visibility methods hide(), show() and toggle() manipulate the CSS display property, and the animated variants fadeIn(), fadeOut(), slideUp(), slideDown() and slideToggle() accept a duration in milliseconds and an optional callback function.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create a webpage with buttons, text boxes and student details.
- Step 2: Bind click, mouse and keyboard events using jQuery.
- Step 3: Change the element style and display messages based on events.
- Step 4: Display typed text whenever a key is pressed.
- Step 5: Hide and show student details using button events.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Event Handling</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

<h2 id="box">Move Mouse Here</h2>

<input type="text" id="input" placeholder="Type something"> <p
id="msg"></p>

<button id="color">Change Colour</button>
<button id="hide">Hide Student</button>
<button id="show">Show Student</button>

<div id="student">
  <p>Name: Rahul</p>
  <p>Roll No: 101</p>
  <p>Branch: CSE</p>
</div>

<script>
$(function () {

  $("#color").on("click", function (e) {
    $(this).css("background", "lightblue");
    $("#box").css("background", "yellow");
  });

  $("#box").on("mouseenter mouseleave", function (e) {
    $("#msg").text(e.type === "mouseenter" ? "Mouse entered!" : "Mouse left!");
  });
});
});
```

```
});

$("#input").on("keyup", function () {
    $("#msg").text($(this).val());
});

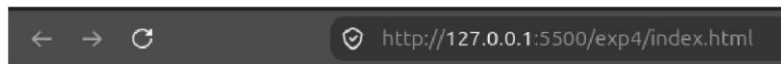
$("#hide").on("click", function () {
    $("#student").hide();
});

$("#show").on("click", function () {
    $("#student").show();
});

});
</script>

</body>
</html>
```

7. OBSERVATION / OUTPUT



Move Mouse Here

Name: Rahul

Roll No: 101

Branch: CSE

After clicking change color button



Move Mouse Here

Mouse left!

Name: Rahul

Roll No: 101

Branch: CSE

After clicking Hide Student button



Move Mouse Here

Mouse left!

Hovering mouse over the text mentioned to do so



Move Mouse Here

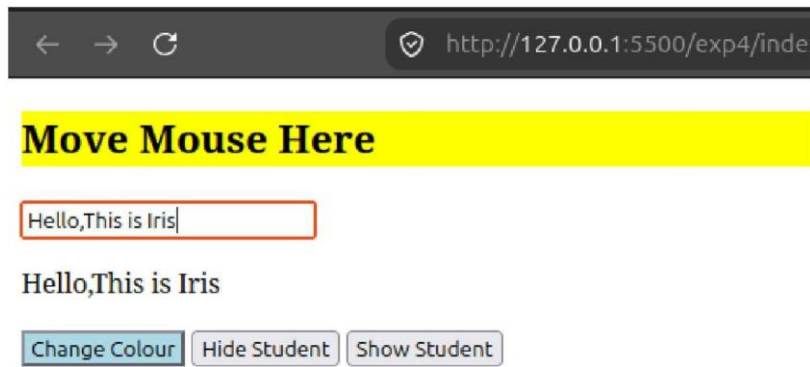
Mouse entered!

Name: Rahul

Roll No: 101

Branch: CSE

Live Typing Tracker worked as follows and Record got hidden after another button click



8. RESULT / INFERENCE

The webpage was created successfully. The background colour was changed on button click, mouse enter and leave messages were displayed, typed text was shown dynamically, and student details were hidden and shown using button events. The events were bound using `on()`, and the event object and `$(this)` were used successfully. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to handle click, mouse and keyboard events using jQuery.
- LO2: Ability to bind events using the `on()` method.
- LO3: Understanding of the event object and its properties.
- LO4: Ability to use `$(this)` to refer to the element that triggered an event.
- LO5: Ability to dynamically modify page content, styling and visibility in response to user events.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
5	5		22

1. TITLE / AIM

Student Registration Page and Student Information Table using Bootstrap

To design a responsive Student Registration Form using the Bootstrap grid, form, button, card and alert components, and to display student information in a responsive Bootstrap table with striped, bordered and hover styles along with contextual highlighting of pass and fail records.

2. OBJECTIVES

- To design a responsive Student Registration Form using Bootstrap.
- To use Bootstrap grid, forms, buttons, cards and alerts.
- To create a responsive Bootstrap table to display student details.
- To apply Bootstrap table styles such as table, table-striped, table-bordered and table-hover.
- To use Bootstrap contextual classes to highlight Pass and Fail students.
- To display a styled table heading and at least five student records. ● To make the table responsive for different screen sizes.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Framework: Bootstrap 5.3 (CSS and bundle JS through a CDN).
- Library: jQuery 3.7.1 (CDN) for the validation message. ● Languages: HTML5, CSS3, JavaScript.

4. THEORY / CONCEPT

Bootstrap is an open-source front-end framework that provides a collection of ready-made CSS classes and JavaScript components for building responsive, mobile-first web pages. Responsiveness is achieved through a twelve-column grid system that is built with flexbox and controlled by CSS media queries.

The grid is constructed from three classes. The container (or container-fluid) class provides the outer wrapper and horizontal padding, the row class creates a horizontal group of columns, and the col-* classes define the width of each column out of twelve. The breakpoint infixes sm, md, lg, xl and xxl correspond to viewport widths of 576 px, 768 px, 992 px, 1200 px and 1400 px, so a column declared as col-md-6 occupies half of the row on medium and larger screens and the full width on smaller screens.

The form components are styled through form-control for input and textarea elements, form-select for drop-down lists, form-check with form-check-input and form-check-label for radio buttons and check boxes, and form-label for labels. Buttons use the btn class together with a variant such as btn-primary or btn-secondary.

The card component groups related content inside a bordered container and consists of card, card-header, card-body, card-title and card-footer. The alert component displays contextual feedback through alert together with alert-success, alert-danger, alert-warning or alert-info, and the class alert-dismissible with a close button allows the message to be removed by the user.

Bootstrap tables are styled through the base class table, with the modifiers table-striped for alternate row shading, table-bordered for borders on all sides, table-hover for a hover highlight and table-sm for a compact layout. Contextual classes such as table-success, table-danger and

table-warning may be applied to an individual row or cell. Wrapping the table in a div with the class table-responsive adds horizontal scrolling on small screens so that the table remains readable on a mobile device.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Include Bootstrap and create a responsive container.
- Step 2: Design the student registration form using Bootstrap grid, card, form and button components.
- Step 3: Add an alert message and registration fields.
- Step 4: Create a responsive table with at least five student records.
- Step 5: Apply striped, bordered and hover styles to the table.
- Step 6: Highlight Pass and Fail records using Bootstrap contextual classes.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
  <link      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container my-4">
  <div class="card p-4 mb-4">
    <h3 class="text-center">Student Registration</h3>
    <div class="alert alert-info">Enter Student Details</div>
    <form>
      <div class="row g-3">
        <div class="col-md-6">
          <input class="form-control" placeholder="Student Name">
        </div>
        <div class="col-md-6">
          <input class="form-control" placeholder="Roll Number">
        </div>
        <div class="col-md-6">
          <input type="date" class="form-control">
        </div>
        <div class="col-md-6">
          <select class="form-select">
            <option>Select Branch</option>
```

```

        <option>CSE</option>
        <option>ECE</option>
        <option>EEE</option>
    </select>
</div>
<div class="col-12">
    <textarea class="form-control" placeholder="Address"></textarea>
</div>
<div class="col-12">
    <button class="btn btn-primary">Submit</button>
    <button type="reset" class="btn btn-secondary">Reset</button>
</div>
</div>
</form>
</div>
<h3 class="text-center">Student Information</h3>
<div class="table-responsive">
    <table class="table table-striped table-bordered table-hover text-center">
        <thead class="table-dark">
            <tr>
                <th>Roll No</th><th>Name</th><th>Branch</th>
                <th>Marks</th><th>Percentage</th><th>Result</th>
            </tr>
        </thead>
        <tbody>
            <tr class="table-
success"><td>101</td><td>Ravi</td><td>CSE</td><td>85</td><td>85%</td><t
d>Pass</td></tr>
            <tr class="table-
danger"><td>102</td><td>Sita</td><td>ECE</td><td>38</td><td>38%</td><td
>Fail</td></tr>
            <tr class="table-
success"><td>103</td><td>Rahul</td><td>CSE</td><td>92</td><td>92%</td><
td>Pass</td></tr>
            <tr class="table-
danger"><td>104</td><td>Anu</td><td>EEE</td><td>35</td><td>35%</td><td
>Fail</td></tr>
            <tr class="table-
success"><td>105</td><td>Arun</td><td>CSE</td><td>78</td><td>78%</td><t
d>Pass</td></tr>

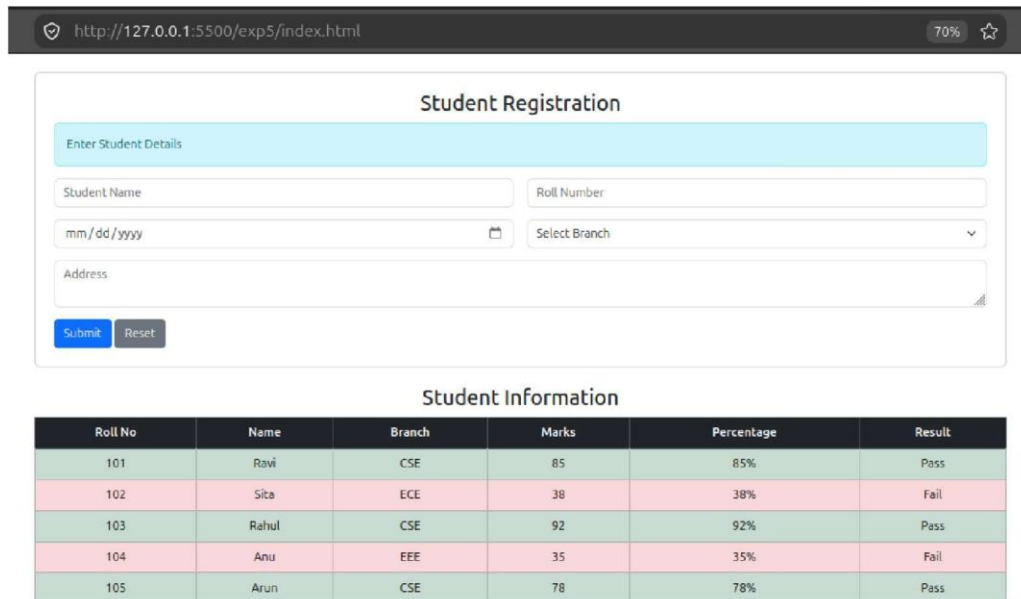
```

```

        </tbody>
    </table>
</div>
</div>
</body>
</html>

```

7. OBSERVATION / OUTPUT



The screenshot shows a web browser at the URL `http://127.0.0.1:5500/exp5/index.html`. The page displays two main sections:

Student Registration

Enter Student Details

Student Name: Roll Number:

mm/dd/yyyy Select Branch:

Address:

Submit Reset

Student Information

Roll No	Name	Branch	Marks	Percentage	Result
101	Ravi	CSE	85	85%	Pass
102	Sita	ECE	38	38%	Fail
103	Rahul	CSE	92	92%	Pass
104	Anu	EEE	35	35%	Fail
105	Arun	CSE	78	78%	Pass

8. RESULT / INFERENCE

The Student Registration and Information webpage was successfully designed using Bootstrap. The registration form was created using Bootstrap grid, forms, cards, buttons and alerts. A responsive student information table containing five records was created using Bootstrap table and contextual classes. Hence, the objectives of the experiment were achieved successfully.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to design responsive web pages using the Bootstrap grid system.
- LO2: Ability to create and style forms using Bootstrap form components.
- LO3: Ability to use Bootstrap cards, buttons and alerts for structured web page design.
- LO4: Ability to create responsive tables using Bootstrap table classes.
- LO5: Ability to apply contextual classes to visually distinguish Pass and Fail student records.
- LO6: Understanding of responsive web design for different screen sizes using Bootstrap.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
6	6		27

- Step 1: Create an AngularJS application with input fields for student details and marks.
- Step 2: Bind the input values using ng-model and display them using ng-bind.
- Step 3: Calculate total and average marks using AngularJS expressions.
- Step 4: Use ng-if to display Pass or Fail based on the marks.
- Step 5: Display all information dynamically on the webpage.
-

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Framework: AngularJS 1.8.3 (CDN).
- Languages: HTML5, CSS3, JavaScript.

4. THEORY / CONCEPT

AngularJS is a structural JavaScript framework that extends HTML with additional attributes called directives and provides data binding between the application data and the HTML view. It allows web pages to update dynamically when the underlying data changes.

A directive is a special attribute that instructs AngularJS to perform a particular operation on a DOM element. The directives and concepts used in this experiment are described below.

- ng-app defines the root element of the AngularJS application and bootstraps the framework.
- ng-model binds an input field to a scope property and provides two-way data binding.
- ng-bind binds the content of an HTML element to a scope property and updates it whenever the property changes.
- ng-if conditionally adds or removes an element from the DOM based on the specified expression.
- AngularJS expressions are used to perform calculations and display dynamic values directly in HTML.

Two-way data binding means that changes made in an input field are automatically reflected in the corresponding model, and changes in the model are automatically reflected in the view. In this experiment, AngularJS expressions are used to calculate the total marks and average marks dynamically whenever the entered marks are changed.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create an AngularJS application with input fields for student details and marks.
- Step 2: Bind the input values using ng-model and display them using ng-bind.
- Step 3: Calculate total and average marks using AngularJS expressions.
- Step 4: Use ng-if to display Pass or Fail based on the marks.
- Step 5: Display all information dynamically on the webpage.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html ng-app>
<head>
  <title>AngularJS Data Binding</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
</head>
<body>

<h2>Student Details</h2>

Name: <input ng-model="name"><br><br>
Branch: <input ng-model="branch"><br><br>

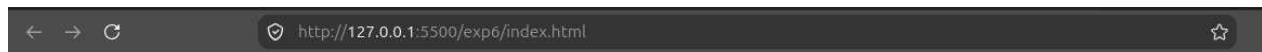
Marks 1: <input type="number" ng-model="m1"><br><br>
Marks 2: <input type="number" ng-model="m2"><br><br>
Marks 3: <input type="number" ng-model="m3"><br><br>

<h3>Student Information</h3>
<p>Name: <span ng-bind="name"></span></p>
<p>Branch: <span ng-bind="branch"></span></p>
<p>Total: {{m1 + m2 + m3}}</p>
<p>Average: {{{(m1 + m2 + m3) / 3}}}</p>

<p ng-if="(m1 + m2 + m3) / 3 >= 40">Result: Pass</p>
<p ng-if="(m1 + m2 + m3) / 3 < 40">Result: Fail</p>

</body>
</html>
```

7. OBSERVATION / OUTPUT



Student Details

Name:

Branch:

Marks 1:

Marks 2:

Marks 3:

Student Information

Name: Iris

Branch: CSE

Total:

Average: NaN



Student Details

Name:

Branch:

Marks 1:

Marks 2:

Marks 3:

Student Information

Name: Iris

Branch: CSE

Total: 272

Average: 90.66666666666667

Result: Pass

8. RESULT / INFERENCE

The AngularJS application was developed successfully. The student's name, branch and marks were accepted through input fields and displayed dynamically using data binding. The total and average marks were calculated using AngularJS expressions, and the result was displayed as Pass or Fail using the ng-if directive. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

LO1: Ability to create and bootstrap an AngularJS application using ng-app, module and controller.

LO2: Ability to implement two-way data binding using ng-model and display values using ng-bind.

LO3: Ability to perform calculations using AngularJS expressions and dynamically display the results.

LO4: Ability to use the ng-if directive to conditionally display content based on application data.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
7	7		31

1. TITLE / AIM

AngularJS Built-in Filters

To develop an AngularJS application that demonstrates the built-in uppercase, lowercase and currency filters by displaying the student name in upper case and lower case and the student fee in currency format, along with the corresponding original values.

2. OBJECTIVES

- To display a student name using the uppercase filter.
- To display a student name using the lowercase filter.
- To display a student's fee using the currency filter.
- To display the original values along with their filtered values.
- To understand the use of AngularJS expressions and built-in filters.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Framework: AngularJS 1.8.3 (CDN).
- Languages: HTML5, CSS3, JavaScript.

4. THEORY / CONCEPT

A filter in AngularJS formats the value of an expression before it is displayed to the user. A filter is applied inside an expression using the pipe symbol, in the form `{{ expression | filterName : argument1 : argument2 }}`. Filters may also be chained, in which case the output of one filter becomes the input of the next, and they may be applied within a directive such as `ng-repeat`.

The built-in filters used in this experiment are the following.

- `uppercase` converts a string to upper case and `lowercase` converts it to lower case.
- `currency` formats a number as a currency amount, prefixing the default symbol of the locale; a symbol and a fraction length may be supplied as arguments, as in `{{ fee | currency : "Rs. " : 2 }}`.
- `number` formats a number with a thousands separator and a specified number of decimal places.
- `date` formats a date object or a timestamp according to a format string such as `"dd-MM-yyyy"`.

- orderBy sorts an array by a specified property, with a leading minus sign indicating descending order.
- limitTo restricts an array or a string to a given number of items or characters.
- filter selects a subset of an array whose items match a given string or object criterion, and is commonly used together with a search text box.

A filter does not modify the underlying model; it only transforms the value on its way to the view. Consequently the original data in the scope remains unchanged and can be displayed alongside the formatted output, which makes the effect of the filter clearly visible. Custom filters may also be registered using the module.filter() method when the built-in set is not sufficient.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create an AngularJS application with student name and fee.
- Step 2: Display the original student name and fee.
- Step 3: Apply uppercase and lowercase filters to the student name.
- Step 4: Apply the currency filter to the student fee.
- Step 5: Display the original and filtered values dynamically.

6. IMPLEMENTATION / SOURCE CODE

```
<!DOCTYPE html>
<html ng-app>
<head>
  <title>AngularJS Built-in Filters</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
</head>
<body>

<h2>Student Details</h2>

Name:
<input type="text" ng-model="name" placeholder="Enter name"><br><br>

Fee:
<input type="number" ng-model="fee" placeholder="Enter fee">

<h3>Output</h3>

<p>Original Name: {{name}}</p>
<p>Uppercase: {{name | uppercase}}</p>
<p>Lowercase: {{name | lowercase}}</p>
```

```
<p>Original Fee: {{fee}}</p>
<p>Currency: {{fee | currency}}</p>
```

```
</body>
</html>
```

7. OBSERVATION / OUTPUT



Student Details

Name:

Fee:

Output

Original Name: Iris Aytas

Uppercase: IRIS AYTAAS

Lowercase: iris aytas

Original Fee: 45000

Currency: \$45,000.00

8. RESULT / INFERENCE

The AngularJS application was developed successfully to demonstrate built-in filters. The uppercase and lowercase filters were used to transform the student name, while the currency filter was used to format the student's fee. The original values were displayed along with their filtered values using AngularJS expressions. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to use AngularJS expressions for displaying dynamic data.
- LO2: Ability to apply the uppercase and lowercase filters to transform string values.
- LO3: Ability to apply the currency filter to format numerical values.
- LO4: Understanding of how AngularJS built-in filters transform data for presentation without changing the original values.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
---------------------------	----------	------	----------

8	8		34
---	---	--	----

1. TITLE / AIM

Student Registration Form using AngularJS

To create a Student Registration Form using AngularJS that binds the form data using ng-model, displays the entered details below the form, verifies that the mandatory fields have been entered, and displays appropriate messages for valid and invalid input.

2. OBJECTIVES

- To bind every form field to the scope using ng-model.
- To display the entered student details below the form as they are typed.
- To verify the mandatory fields using the required attribute and the AngularJS form controller.
- To validate the email address, the mobile number pattern and the numeric range of the marks.
- To display validation messages for invalid input and a confirmation message for valid input.
- To disable the submit button until the form becomes valid.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code.
- Framework: AngularJS 1.8.3 (CDN), Bootstrap 5.3 (CDN) for styling.
- Languages: HTML5, CSS3, JavaScript.

4. THEORY / CONCEPT

AngularJS extends the standard HTML form with a form controller that tracks the state of every input element bound through ng-model. When a form is given a name attribute inside an ng-app region, AngularJS publishes an object of that name on the scope, and each named input becomes a property of that object. The state of a control is exposed through a set of boolean properties.

- \$valid and \$invalid indicate whether the control satisfies all of its validators.
- \$pristine and \$dirty indicate whether the value has been modified by the user.
- \$touched and \$untouched indicate whether the control has been visited and left.
- \$error is an object whose keys are the failed validators, such as required, email, pattern, minlength and number.

The validation directives are written as ordinary HTML attributes. The required attribute makes a field mandatory, the input type email applies the built-in email validator, ng-pattern applies a regular expression, ng-minlength and ng-maxlength restrict the length, and min and max restrict

a numeric range. AngularJS also adds the CSS classes ng-valid, ng-invalid, ng-pristine, ng-dirty, ng-touched and ng-untouched to the element, which allows the invalid fields to be styled through the stylesheet.

A validation message is displayed by combining ng-show with the state of the control, for example ng-show="regForm.email.\$invalid && regForm.email.\$touched". Combining the \$touched condition prevents an error message from appearing before the user has visited the field. The submit button is normally disabled through ng-disabled="regForm.\$invalid" so that an incomplete form cannot be submitted, and the novalidate attribute is placed on the form element to suppress the default validation performed by the browser.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create an AngularJS registration form with fields for student details.
- Step 2: Bind all fields using ng-model and apply required validations.
- Step 3: Validate email, mobile number and marks using AngularJS validation.
- Step 4: Display entered details dynamically below the form.
- Step 5: Disable the submit button until all fields are valid.
- Step 6: Display appropriate validation or confirmation messages.

6. IMPLEMENTATION / SOURCE CODE

```
<<!DOCTYPE html>
<html ng-app="app">
<head>
  <title>Student Registration</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
</head>
<body ng-controller="ctrl">

  <h2>Student Registration Form</h2>

  <form name="f" ng-submit="submitted=true" novalidate>

    Name:
    <input type="text" name="name" ng-model="s.name" required><br><br>
    Roll No:
    <input type="text" name="roll" ng-model="s.roll" required><br><br>

    Branch:
    <input type="text" name="branch" ng-model="s.branch" required><br><br>
```

Email:

```
<input type="email" name="email" ng-model="s.email" required><br><br>
```

Mobile:

```
<input type="text" name="mobile" ng-model="s.mobile" ng-pattern="/^[0-9]{10}$/" required><br><br>
```

Marks:

```
<input type="number" name="marks" ng-model="s.marks"
      min="0" max="100" required><br><br>
```

```
<button type="submit" ng-disabled="f.$invalid">Submit</button>
</form>
```

```
<p ng-if="f.$invalid">Please enter valid details.</p>
```

```
<p ng-if="submitted && f.$valid">Registration successful!</p>
```

```
<h3>Student Details</h3>
```

```
<p>Name: {{s.name}}</p> <p>Roll
```

```
No: {{s.roll}}</p>
```

```
<p>Branch: {{s.branch}}</p>
```

```
<p>Email: {{s.email}}</p>
```

```
<p>Mobile: {{s.mobile}}</p>
```

```
<p>Marks: {{s.marks}}</p>
```

```
<script>
```

```
angular.module("app", [])
```

```
.controller("ctrl", function($scope) {
```

```
  $scope.s = {};
```

```
  $scope.submitted = false;
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

7. OBSERVATION / OUTPUT**Student Registration Form**Name: Roll No: Branch: Email: Mobile: Marks:

Please enter valid details.

Student Details

Name:

Roll No:

Branch:

Email:

Mobile:

Marks:

As the form began to fill up...



Student Registration Form

Name:

Roll No:

Branch:

Email:

Mobile:

Marks:

Please enter valid details.

Student Details

Name: Iris Aytas

Roll No: 081

Branch: CS

Email:

Mobile:

Marks:

8. RESULT / INFERENCE

The AngularJS Student Registration Form was developed successfully. Form data was bound using ng-model, entered student details were displayed dynamically below the form, mandatory fields and input formats were validated using the AngularJS form controller, and appropriate validation and confirmation messages were displayed. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to implement form data binding using ng-model.
- LO2: Ability to display dynamically entered student details using AngularJS expressions.
- LO3: Ability to validate mandatory fields and input formats using the AngularJS form controller.
- LO4: Ability to display appropriate messages for valid and invalid form inputs.
- LO5: Ability to control form submission using \$valid and reset the form using \$setPristine() and \$setUntouched().

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
---------------------------	----------	------	----------

9	9		39
---	---	--	----

1. TITLE / AIM**AngularJS Service and Routing Application**

To develop a Student Information Application containing Home, Student List, Student Details and Login views, to navigate between the views using AngularJS routing, and to supply the student data to the views through a custom AngularJS service.

2. OBJECTIVES

- To configure the ngRoute module and define routes for the Home, Student List, Student Details and Login views.
- To create a custom AngularJS service that provides the student information.
- To inject the service into the controllers and display the student data in the Student List view.
- To pass a route parameter and display the corresponding record in the Student Details view.
- To implement a simple Login view that stores the login state in the service.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Google Chrome / Mozilla Firefox, Visual Studio Code, Live Server extension (or any local HTTP server).
- Framework: AngularJS 1.8.3 with angular-route 1.8.3 (CDN), Bootstrap 5.3 (CDN).
● Languages: HTML5, CSS3, JavaScript.

4. THEORY / CONCEPT

Routing allows a single page application to display different views inside the same page without a full page reload. AngularJS provides routing through the optional ngRoute module, which must be loaded separately as angular-route.js and listed as a dependency of the application module. The \$routeProvider service is configured inside the config block of the module, where each route is registered using when(path, route) and a fallback is registered using otherwise(route).

Each route definition specifies the templateUrl of the partial HTML file to be loaded and the controller that manages it. The directive ng-view marks the region of the page in which the matching template is rendered. The URL of a route is written after the hash symbol, for example index.html#!/students, so the browser does not send a fresh request to the server. A route may

contain a parameter written as `/student/:roll`, and the value supplied in the URL is read inside the controller through the `$routeParams` service.

A service in AngularJS is a singleton object that holds reusable business logic or data and is shared by all the controllers into which it is injected. The three common ways of creating one are the `factory()` method, which returns an object, the `service()` method, which is instantiated with the `new` keyword, and the `value()` and `constant()` methods for simple values. Because the service is a singleton, the data placed in it survives navigation between views, which makes it the natural place to keep the student records and the login state.

Dependency injection is the mechanism by which AngularJS supplies the required services to a controller. When a controller function declares a parameter named `StudentService`, the injector locates the registered provider of that name and passes the instance to the function, which removes the need for the controller to create its own dependencies.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Include `angular.min.js` and `angular-route.min.js`, and declare the module `studentApp` with `ngRoute` as a dependency.
- Step 2: Configure `$routeProvider` with the routes `/home`, `/students`, `/student/:roll` and `/login`, and set `otherwise` to redirect to `/home`.
- Step 3: Create the partial templates `home.html`, `students.html`, `details.html` and `login.html`.
- Step 4: Place the `ng-view` directive in `index.html` and add a navigation bar containing anchor tags that point to the route URLs.
- Step 5: Create a factory service named `StudentService` that holds an array of student objects and exposes the methods `getAll()`, `getByRoll(roll)`, `login(user, password)` and `isLoggedIn()`.
- Step 6: Inject `StudentService` into the list controller and assign the array returned by `getAll()` to the scope.
- Step 7: Display the array in the Student List view using `ng-repeat`, with a link on each row pointing to the detail route of that roll number.
- Step 8: Inject `$routeParams` into the details controller, read the roll number and fetch the matching record using `getByRoll()`.
- Step 9: Implement the login controller so that it validates the credentials through the service and redirects to `/students` using `$location.path()` on success.
- Step 10: Serve the folder through a local HTTP server, because `templateUrl` requests are blocked by the browser when the page is opened directly from the file system.

6. IMPLEMENTATION / SOURCE CODE

index.html

```
<!DOCTYPE html>
```

```
<html lang="en" ng-app="studentApp">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Information Application</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular-
route.min.js"></script>

  <script> var app = angular.module("studentApp",
    ["ngRoute"]);

    app.config(function($routeProvider) {
      $routeProvider
        .when("/home", {
          templateUrl: "home.html",
          controller: "HomeCtrl"
        })
        .when("/students", {
          templateUrl: "students.html",
          controller: "StudentListCtrl"
        })
        .when("/student/:roll", {
          templateUrl: "details.html",
          controller: "StudentDetailsCtrl"
        })
        .when("/login", {
          templateUrl: "login.html",
          controller: "LoginCtrl"
        })
        .otherwise({
          redirectTo: "/home"
        });
    }); app.factory("StudentService",

    function() {
```

```
var students = [  
  {  
    roll: "101", name:  
    "Rahul Kumar",  
    branch: "CSE",  
    marks: 85  
  },  
  {  
    roll: "102", name:  
    "Priya Sharma",  
    branch: "ECE",  
    marks: 91  
  },  
  {  
    roll: "103", name:  
    "Arjun Reddy",  
    branch: "EEE",  
    marks: 78  
  },  
  {  
    roll: "104", name:  
    "Sneha Rao",  
    branch: "IT",  
    marks: 88  
  },  
  {  
    roll: "105", name:  
    "Vikram Singh",  
    branch: "MECH",  
    marks: 72  
  }  
];  
  
var loggedIn = false;  
  
return {  
  
  getAll: function() { return  
    students;  
  },  
}
```

```
getByRoll: function(roll) { for (var i = 0; i <
    students.length; i++) { if
    (students[i].roll === roll) { return
    students[i];
    }
    }
    return null;
},

login: function(user, password) {
    if (user === "admin" && password === "1234") {
        loggedIn = true;
        return true;
    }
    return false;
},

isLoggedIn: function() {
    return loggedIn;
}
}; });

app.controller("HomeCtrl", function($scope, StudentService) {
    $scope.loggedIn = StudentService.isLoggedIn();
});

app.controller("StudentListCtrl", function($scope, StudentService) {
    $scope.students = StudentService.getAll();
});

app.controller("StudentDetailsCtrl", function($scope, $routeParams, StudentService) {
    $scope.student = StudentService.getByRoll($routeParams.roll);
}); app.controller("LoginCtrl", function($scope, $location,

StudentService) { $scope.login = function() { if

(StudentService.login($scope.username, $scope.password)) {

    $scope.message = "Login successful!";
    $location.path("/students");
```

```
        } else {
            $scope.message = "Invalid username or password.";
        }
    };
});
</script>
</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="#!/home">Student App</a>

        <div>
            <a class="btn btn-outline-light me-2" href="#!/home">Home</a>
            <a class="btn btn-outline-light me-2" href="#!/students">Student List</a>
            <a class="btn btn-outline-light" href="#!/login">Login</a> </div>
        </div>
    </nav>

    <div class="container mt-4">
        <div ng-view></div>
    </div>

</body>
</html>
```

home.html

```
<div class="text-center">
    <h2>Student Information Application</h2>
    <p class="lead">Welcome to the Student Information System.</p>

    <a href="#!/students" class="btn btn-primary">View Students</a>
    <a href="#!/login" class="btn btn-success">Login</a>
</div>
```

students.html

```
<div>
    <h2>Student List</h2>

    <table class="table table-bordered table-striped">
```

```
<thead class="table-dark">
  <tr>
    <th>Roll Number</th>
    <th>Name</th>
    <th>Branch</th>
    <th>Marks</th>
    <th>Details</th>
  </tr>
</thead>

<tbody>
  <tr ng-repeat="student in students">
    <td>{{ student.roll }}</td>
    <td>{{ student.name }}</td>
    <td>{{ student.branch }}</td>
    <td>{{ student.marks }}</td>
    <td>
      <a href="#!/student/{{ student.roll }}"
        class="btn btn-sm btn-primary">
        View Details
      </a>
    </td>
  </tr>
</tbody>
</table>
</div>
details.html
<div>
  <h2>Student Details</h2>

  <div ng-if="student">
    <div class="card">
      <div class="card-body">
        <p><strong>Roll Number:</strong> {{ student.roll }}</p>
        <p><strong>Name:</strong> {{ student.name }}</p>
        <p><strong>Branch:</strong> {{ student.branch }}</p>
        <p><strong>Marks:</strong> {{ student.marks }}</p>
      </div>
    </div>
  </div>
</div>
```

```
<div ng-if="!student" class="alert alert-danger">
  Student not found.
</div>
```

```
<br>
```

```
<a href="#!/students" class="btn btn-secondary">
  Back to Student List
</a>
</div>
```

login.html

```
<div class="row justify-content-center">
  <div class="col-md-5">

    <h2 class="text-center">Login</h2>

    <form ng-submit="login()">

      <div class="mb-3">
        <label class="form-label">Username</label>
        <input type="text" class="form-
          control" ng-model="username"
          required>
      </div>

      <div class="mb-3">
        <label class="form-label">Password</label>
        <input type="password"
          class="form-control" ng-
          model="password" required>
      </div>

      <button type="submit" class="btn btn-primary">
        Login
      </button>

    </form>

    <p class="mt-3" ng-if="message">
      {{ message }}
    </p>
```

```
<p class="text-muted">
  Username: admin<br>
  Password: 1234
</p>

</div>
</div>
```

7. OBSERVATION / OUTPUT



Student Information Application

Welcome to the Student Information System.

[View Students](#) [Login](#)

After clicking View Students button



Student List

Roll Number	Name	Branch	Marks	Details
101	Rahul Kumar	CSE	85	View Details
102	Priya Sharma	ECE	91	View Details
103	Arjun Reddy	EEE	78	View Details
104	Sneha Rao	IT	88	View Details
105	Vikram Singh	MECH	72	View Details

After clicking View Details button

Student App

Home Student List Login

Student Details

Roll Number: 102

Name: Priya Sharma

Branch: ECE

Marks: 91

Back to Student List

After clicking Login button

Student App

Home Student List Login

Login

Username

Password

Login

Username: admin
Password: 1234

After entering the provided credentials and Logged in

Student App

Home Student List Login

Save password for http://127.0.0.1:5500/?
Username: admin
Password:
Not now Save

Student List

Roll Number	Name	Branch	Marks	Details
101			85	View Details
102	Priya Sharma	ECE	91	View Details
103	Arjun Reddy	EEE	78	View Details
104	Sneha Rao	IT	88	View Details
105	Vikram Singh	MECH	72	View Details

8. RESULT / INFERENCE

The Student Information Application was successfully developed using AngularJS routing and a custom service. The application navigates between multiple views, retrieves student information through StudentService, displays the student list dynamically, shows individual student details using route parameters, and maintains the login state through the service.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to configure client-side routing using the ngRoute module, \$routeProvider, templateUrl and the ng-view directive.
- LO2: Ability to create a reusable singleton service using factory() and to share data and state between controllers through dependency injection.
- LO3: Ability to read route parameters using \$routeParams and to navigate programmatically using \$location.path().

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
10	10		50

1. TITLE / AIM

Node.js Built-in Modules - fs, http, path, os and events

To develop a Node.js application that demonstrates the basic usage of the built-in modules fs, http, path, os and events by writing and reading a student information file, creating an HTTP server, extracting file name and extension, displaying operating system information and raising a custom event.

2. OBJECTIVES

- To create a text file containing student information and read its contents using the fs module.
- To create a basic HTTP server that displays a welcome message in the browser using the http module.
- To display the file name and the file extension of a given path using the path module.
- To display the platform, CPU architecture and total memory using the os module. ● To create and trigger a custom event named welcome using the events module.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Node.js 20.x LTS, npm, Visual Studio Code, Google Chrome.
- Modules: fs, http, path, os and events (all built into the Node.js runtime). ● Languages: JavaScript (CommonJS).

4. THEORY / CONCEPT

Node.js is a run-time environment that executes JavaScript outside the browser using the V8 engine. It follows a single-threaded, event-driven, non-blocking input and output model, in which a long-running operation is handed over to the system and a callback is executed when the operation completes. A module in Node.js is a reusable unit of code loaded through the require() function; the built-in or core modules are compiled into the binary and need no installation.

The core modules used in this experiment serve the following purposes.

- fs provides file system operations. The synchronous forms writeFileSync(), readFileSync() and appendFileSync() block until they finish, whereas the asynchronous forms writeFile(), readFile() and appendFile() accept a callback of the form (err, data).

- http provides the facility to create a server through `createServer(callback)` and to listen on a port through `listen(port)`. The callback receives a request object and a response object, and the response is completed with `writeHead()`, `write()` and `end()`.
- path provides utilities for handling file paths, including `basename()`, `dirname()`, `join()` and `parse()`; these work correctly on every operating system because the module is aware of the platform separator.
- os provides information about the underlying operating system through `platform()`, `arch()`, `totalmem()`, `freemem()`, `cpus()`, `hostname()` and `type()`.
- events provides the `EventEmitter` class, which implements the publisher-subscriber pattern. A listener is registered through `on(eventName, handler)` and the event is raised through `emit(eventName, arguments)`.

The event driven nature of Node.js is central to its design: almost every core object that emits notifications, including streams and servers, is an instance of `EventEmitter`, so the pattern demonstrated with the custom welcome event is the same mechanism used internally by the framework.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create a project folder and a file named `app.js`, and import the five core modules using `require()`.
- Step 2: Prepare a string containing student information and write it to `student.txt` using `fs.writeFileSync()`.
- Step 3: Append an additional record using `fs.appendFileSync()` and read the whole file back using `fs.readFile()` with a callback, then print the contents.
- Step 4: Use `path.basename()`, `path.dirname()` and `path.parse()` on the path of the created file and print the results.
- Step 5: Use `os.platform()`, `os.arch()`, `os.totalmem()`, `os.freemem()`, `os.cpus().length` and `os.hostname()` and print the values, converting the memory from bytes to gigabytes.
- Step 6: Create an instance of `EventEmitter`, register a listener for a custom event named `welcome`, and raise the event using `emit()` with an argument.
- Step 7: Create an HTTP server using `http.createServer()`, send a welcome message with the status code 200 and the content type `text/html`, and make it listen on port 3000.
- Step 8: Run the program using the command `node app.js` and observe the console output.
- Step 9: Open `http://localhost:3000` in the browser and verify the welcome message.

6. IMPLEMENTATION / SOURCE CODE

```
const fs = require("fs");  
const http = require("http");
```

```
const path =
require("path"); const os =
require("os"); const
EventEmitter =
require("events");

// 1. fs Module - Write and Read Student Information
const studentInfo = "Name: Rahul Kumar\nRoll No: 101\nBranch: CSE";

fs.writeFileSync("student.txt", studentInfo); const data =

fs.readFileSync("student.txt", "utf8");

console.log("Student Information:");
console.log(data);

// 2. path Module - File Name and Extension const
filePath = path.join(__dirname, "student.txt");

console.log("\nPath Information:"); console.log("File
Name:", path.basename(filePath)); console.log("File
Extension:", path.extname(filePath));

// 3. os Module - Operating System Information
console.log("\nOperating System Information:");
console.log("Platform:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory:", (os.totalmem() / (1024 ** 3)).toFixed(2), "GB");

// 4. events Module - Custom Event const
eventEmitter = new EventEmitter();

eventEmitter.on("welcome", function(name) {
  console.log("\nWelcome event triggered.");
  console.log("Welcome, " + name + "!");
}); eventEmitter.emit("welcome",

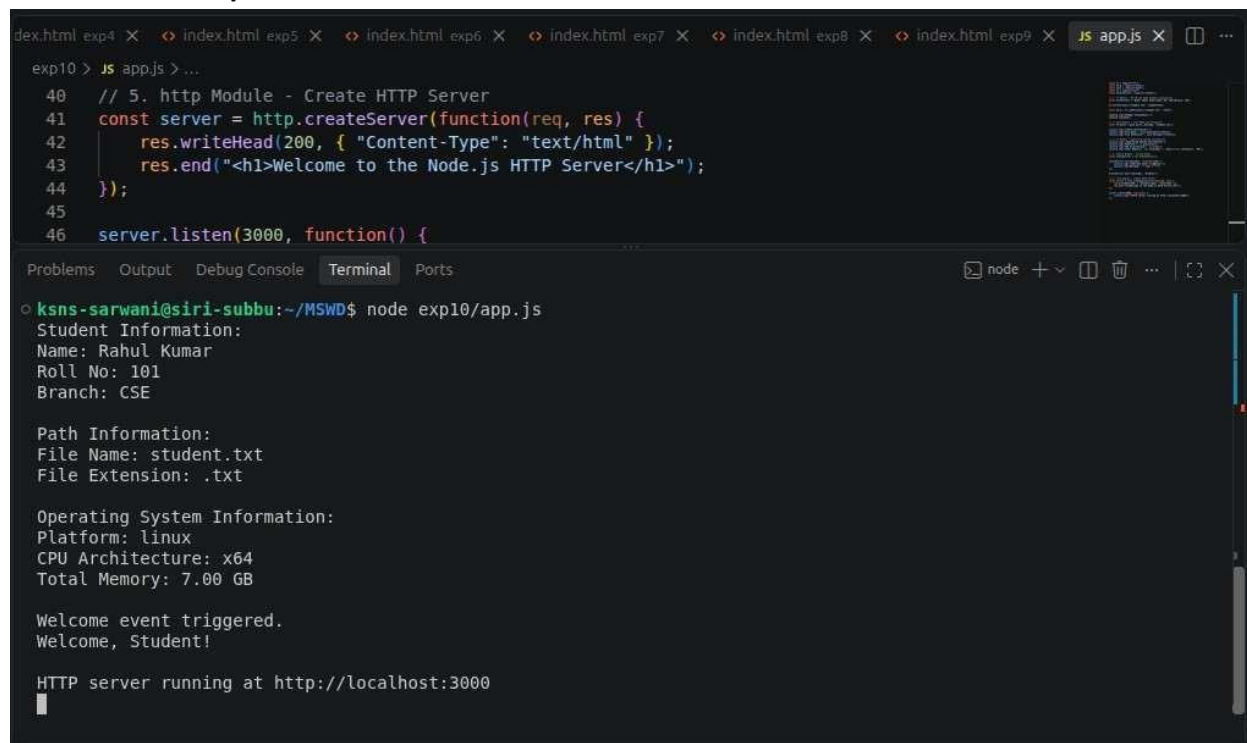
"Student");
```

```
// 5. http Module - Create HTTP Server const
server = http.createServer(function(req, res) {
  res.writeHead(200, { "Content-Type": "text/html" });
  res.end("<h1>Welcome to the Node.js HTTP Server</h1>");
});

server.listen(3000, function() {
  console.log("\nHTTP server running at http://localhost:3000");
});
```

And executed a command : node app.js

7. OBSERVATION / OUTPUT



The screenshot shows a VS Code editor with a file named `app.js` containing the following code:

```
40 // 5. http Module - Create HTTP Server
41 const server = http.createServer(function(req, res) {
42   res.writeHead(200, { "Content-Type": "text/html" });
43   res.end("<h1>Welcome to the Node.js HTTP Server</h1>");
44 });
45
46 server.listen(3000, function() {
```

The terminal output shows the command `node exp10/app.js` being executed, followed by student information, path information, operating system information, and the message "Welcome event triggered. Welcome, Student!". The terminal also shows "HTTP server running at http://localhost:3000".



Welcome to the Node.js HTTP Server

8. RESULT / INFERENCE

The Node.js application was successfully developed using the built-in fs, http, path, os and events modules. File operations, HTTP server creation, path handling, operating system information retrieval and custom event handling were successfully demonstrated.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to perform basic file creation, writing and reading operations using the fs module.
- LO2: Ability to create a basic HTTP server using the http module and handle browser requests.
- LO3: Ability to work with file paths and retrieve file names and extensions using the path module.
- LO4: Ability to retrieve basic operating system information using the os module.
- LO5: Ability to create, register and trigger custom events using the EventEmitter class.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
11	11		55

1. TITLE / AIM

Create user-defined Node.js modules for performing arithmetic operations and calculating student total marks and percentage, and use these modules in app.js to accept user input and display the results.

2. OBJECTIVES

- To create user-defined modules calculator.js and student.js.
- To define and export functions from the modules.
- To import and use the modules in app.js.
- To accept user input using Node.js.
- To perform arithmetic operations and calculate student results.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Node.js 20.x LTS, npm, Visual Studio Code, Google Chrome.
- Modules: fs, http, path, os and events (all built into the Node.js runtime). ● Languages: JavaScript (CommonJS).

4. THEORY / CONCEPT

Node.js follows the CommonJS module system, in which every file is treated as a separate module with its own private scope. A variable or function declared inside a file is not visible outside it unless it is explicitly exported, which prevents accidental pollution of the global namespace and allows the same name to be used independently in different files.

Exporting is performed through the object `module.exports`, which is the value returned by `require()`. There are three common forms.

- Attaching properties one by one, as in `exports.add = function (a, b) { return a + b; };`
- Assigning a complete object, as in `module.exports = { add, sub, mul, div };`
- Assigning a single function or a class, as in `module.exports = Calculator;` which makes the module itself callable.

Importing is performed through `require("./calculator")`, where the leading `./` indicates a local file rather than a core module or a package installed under `node_modules`. The extension `.js` may be omitted. Node.js caches a module after the first `require()`, so the file is executed only once even when it is required from several places, and the same exported object is shared.

The readline core module is used to read input from the console. An interface is created through `readline.createInterface({ input: process.stdin, output: process.stdout })`, and the method `question(prompt, callback)` reads one line at a time. Since these calls are asynchronous, the subsequent prompts are nested inside the callbacks, and the interface is finally closed through `close()`. Alternatively the values may be supplied as command line arguments and read from the array `process.argv`.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create `calculator.js` with functions for addition, subtraction, multiplication and division.
- Step 2: Create `student.js` with functions for total marks and percentage.
- Step 3: Export the functions from both modules.
- Step 4: Import the modules into `app.js`.
- Step 5: Accept two numbers and three subject marks from the user. ● Step 6: Perform the operations and display the results.

6. IMPLEMENTATION / SOURCE CODE

calculator.js

```
exports.add = (a, b) => a + b;  
exports.sub = (a, b) => a - b;  
exports.mul = (a, b) => a * b;  
exports.div = (a, b) => a / b;
```

student.js

```
exports.total = (a, b, c) => a + b + c;  
exports.percentage = (a, b, c) => (a + b + c) / 3;
```

```
app.js const readline =  
require("readline"); const calc =  
require("./calculator"); const  
student = require("./student");
```

```
const r =  
  readline.createInterface({ input:  
    process.stdin, output:  
    process.stdout  
  });
```

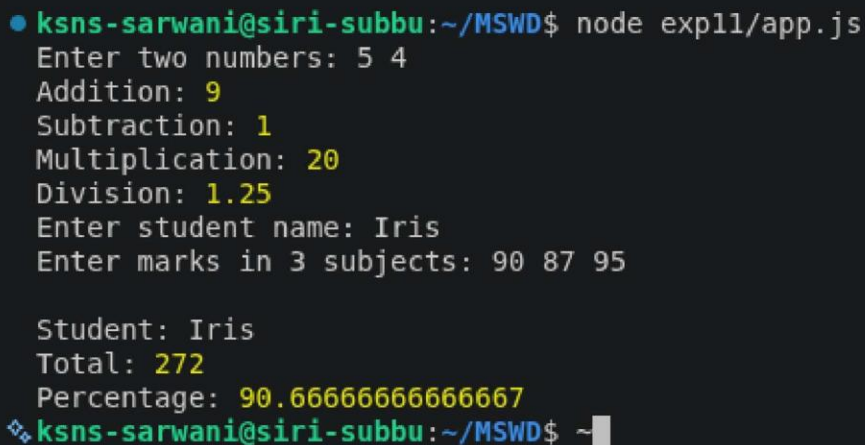
```
r.question("Enter two numbers: ", x => {
  let [a, b] = x.split(" ").map(Number);
  console.log("Addition:", calc.add(a, b));
  console.log("Subtraction:", calc.sub(a,
  b)); console.log("Multiplication:",
  calc.mul(a, b)); console.log("Division:",
  calc.div(a, b));

  r.question("Enter student name: ", name => {
    r.question("Enter marks in 3 subjects: ", m => {
      let [x, y, z] = m.split(" ").map(Number);

      console.log("\nStudent:", name); console.log("Total:",
      student.total(x, y, z)); console.log("Percentage:",
      student.percentage(x, y, z));

      r.close();
    });
  });
});
```

7. OBSERVATION / OUTPUT



```
ksns-sarwani@siri-subbu:~/MSWD$ node exp11/app.js
Enter two numbers: 5 4
Addition: 9
Subtraction: 1
Multiplication: 20
Division: 1.25
Enter student name: Iris
Enter marks in 3 subjects: 90 87 95

Student: Iris
Total: 272
Percentage: 90.66666666666667
ksns-sarwani@siri-subbu:~/MSWD$ ~
```

8. RESULT / INFERENCE

The user-defined modules calculator.js and student.js were created successfully and their functions were exported using module.exports and the exports shorthand. The modules were imported into app.js using require(), two numbers were accepted and the results of addition, subtraction, multiplication, division and modulus were displayed, and the total marks,

percentage, grade and result of a student were computed and displayed. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to organise an application into user-defined modules and to export functions through `module.exports` and `exports`.
- LO2: Ability to import a local module using `require()` with a relative path and to understand the module caching behaviour of Node.js.
- LO3: Ability to read input from the console using the `readline` module and to handle the nesting of asynchronous callbacks.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
12	12		59

1. TITLE / AIM

Node.js Event Emitter and Streams

To develop a Node.js application for processing a large student data file by creating a custom event named `studentAdded`, generating and handling it using the `EventEmitter` class, and reading the student data in chunks using a readable stream.

2. OBJECTIVES

- To create a custom event named `studentAdded` using the `EventEmitter` class.
- To register a listener for the event and to raise the event using `emit()`.
- To create a readable stream for a large student data file.
- To read and display the data in chunks using the `data` event of the stream.
- To handle the `end` and `error` events of the stream and to count the records processed.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Node.js 20.x LTS, Visual Studio Code.
- Modules: `events`, `fs` and `readline` (core modules).
- Data file: `students.csv` containing a large number of student records.
- Languages: JavaScript (CommonJS).

4. THEORY / CONCEPT

The `EventEmitter` class provided by the `events` module implements the publisher-subscriber pattern in Node.js. A listener is registered for a named event using `on(name, handler)` or `once(name, handler)`, and the event is raised using `emit(name, arg1, arg2, ...)`. Every argument passed to `emit()` is forwarded to the handlers, which are executed synchronously in the order of registration. Additional methods include `removeListener()`, `listenerCount()` and the special `error` event, which throws an exception when it is emitted without a registered listener.

A stream is an abstraction for reading or writing data piece by piece rather than loading it into memory as a whole. This is essential when a file is large, because reading the entire file with `readFileSync()` would place the whole content in memory at once. Node.js provides four kinds of streams: readable streams for input, writable streams for output, duplex streams for both directions, and transform streams that modify the data as it passes through.

A readable stream is created through `fs.createReadStream(path, options)`, where the options may specify the encoding and the `highWaterMark`, which is the size of the internal buffer and

therefore the approximate size of each chunk. Since every stream is itself an EventEmitter, the data is consumed by attaching handlers to its events.

- The data event is raised for every chunk read from the source.
- The end event is raised when the source has no further data.
- The error event is raised when the read operation fails.
- The pipe() method connects a readable stream to a writable stream and automatically manages the flow of data, a mechanism known as back pressure.

Because the chunk boundary of a readable stream does not necessarily coincide with a line boundary, the readline module is used with the stream when the data must be processed line by line, as is the case for a comma separated file of student records.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Import EventEmitter and fs modules.
- Step 2: Create an EventEmitter object and register the studentAdded event.
- Step 3: Emit the event with student details and handle it using a listener.
- Step 4: Create a readable stream for the student data file.
- Step 5: Read and display the file data chunk by chunk.
- Step 6: Handle end and error events and count the processed records.

6. IMPLEMENTATION / SOURCE CODE

students.txt 101,Ravi,CSE

102,Sita,ECE

103,Rahul,EEE

104,Anu,CSE 105,Arun,IT

app.js

```
const fs = require("fs");
```

```
const EventEmitter = require("events");
```

```
const event = new EventEmitter(); let
```

```
count = 0;
```

```
event.on("studentAdded", student => {  
  console.log("Student Added:", student); });
```

```
event.emit("studentAdded", "106,John,CSE"); const stream =
```

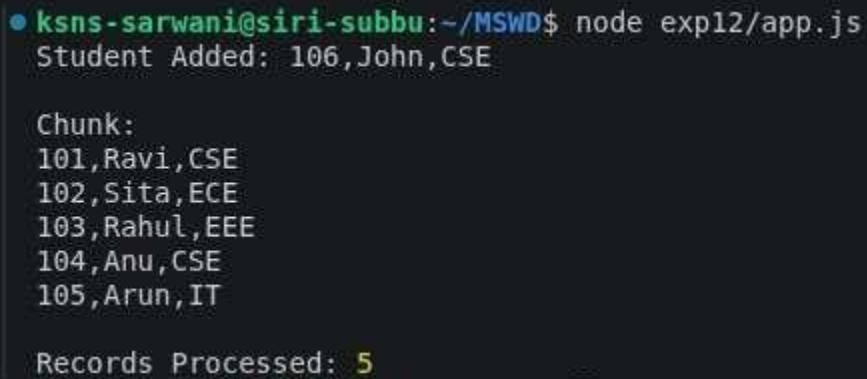
```
fs.createReadStream(__dirname + "/students.txt", "utf8");
```

```
stream.on("data", chunk => {  
  console.log("\nChunk:\n" + chunk);  
  count += chunk.split("\n").filter(x => x.trim()).length;  
});
```

```
stream.on("end", () => { console.log("\nRecords  
  Processed:", count);  
});
```

```
stream.on("error", err => { console.log("Error:",  
  err.message);  
});
```

7. OBSERVATION / OUTPUT



```
● ksns-sarwani@siri-subbu:~/MSWD$ node exp12/app.js  
Student Added: 106,John,CSE  
  
Chunk:  
101,Ravi,CSE  
102,Sita,ECE  
103,Rahul,EEE  
104,Anu,CSE  
105,Arun,IT  
  
Records Processed: 5
```

8. RESULT / INFERENCE

The Node.js application for processing a large student data file was developed successfully. A custom event named studentAdded was created using the EventEmitter class and was raised once for every record and handled by the registered listener. The student data was read using a readable stream and displayed in chunks, the end and error events were handled, and the summary of the records processed was displayed. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to define an application class that extends EventEmitter and to raise and handle custom events using emit() and on().
- LO2: Ability to read a large file incrementally using a readable stream and to handle its data, end and error events.

- LO3: Understanding of the highWaterMark buffer size, chunk boundaries and the advantage of streaming over loading a complete file into memory.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
13	13		63

1. TITLE / AIM

Student Information Web Server using Node.js

To develop a simple Student Information web server using the built-in http module of Node.js, which displays a welcome message on the home page, a list of students, the details of an individual student, and the student information as a JSON response, using different URLs for each operation.

2. OBJECTIVES

- To create a web server using the http module and make it listen on a port.
- To display a welcome message on the home page.
- To display a list of students on a separate URL.
- To display the details of one student on another URL.
- To send the student information as a JSON response.
- To handle an unknown URL with the status code 404.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Node.js 20.x LTS, Visual Studio Code, Google Chrome.
- Modules: http and url (core modules).
- Languages: JavaScript (CommonJS), HTML.

4. THEORY / CONCEPT

The http core module allows a Node.js program to act as a web server without any external framework. The server is created through `http.createServer(callback)`, where the callback is executed once for every incoming request and receives two objects. The request object, an instance of `IncomingMessage`, exposes the properties `url`, `method` and `headers`. The response object, an instance of `ServerResponse`, is used to construct the reply through `writeHead(statusCode, headers)`, `write(data)` and `end()`.

Routing is the process of selecting the action to be taken according to the requested URL and HTTP method. Since the http module performs no routing of its own, it is implemented manually by examining `req.url` inside the callback, usually with a switch statement or a chain of conditions. The url core module, or the WHATWG URL class, is used when the path has to be separated from the query string, as in `/student?roll=21B81A0501`.

The Content-Type header informs the client how the body of the response is to be interpreted. The value text/html causes the browser to render the response as a web page, text/plain displays it as plain text, and application/json indicates a JSON payload, which is produced by passing a JavaScript object to JSON.stringify(). A status code of 200 indicates success, 404 indicates that the resource was not found and 500 indicates a server error.

The server begins to accept connections when listen(port, callback) is called. Because Node.js is single threaded and event driven, one process is able to serve many concurrent connections, since each request handler returns immediately and any slow operation is performed asynchronously.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Import the Node.js http module and create student data.
- Step 2: Create an HTTP server and check the requested URL.
- Step 3: Display the welcome message, student list and individual student details for different URLs.
- Step 4: Send student data as JSON for the API URL.
- Step 5: Return status code 404 for unknown URLs. ● Step 6: Start the server on a specified port.

6. IMPLEMENTATION / SOURCE CODE7.

OBSERVATION / OUTPUT



Welcome to Student Information Server

url-http://localhost:3000/students



Student List

1 - Ravi

2 - Sita

3 - Rahul

url-http://localhost:3000/student/1

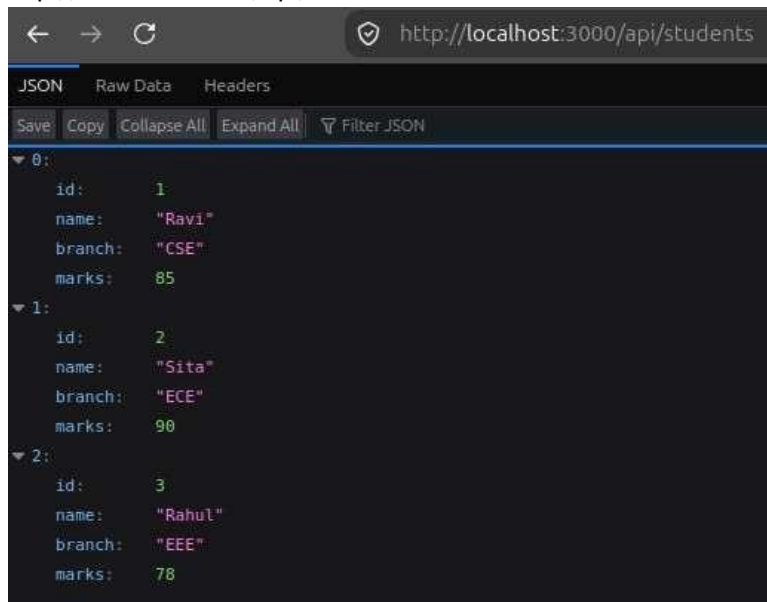


Ravi

Branch: CSE

Marks: 85

http://localhost:3000/api/students



8. RESULT / INFERENCE

The Student Information web server was successfully created and executed using Node.js. The home page displayed the welcome message, the /students URL displayed the list of three students, and the /student/1 URL displayed the details of Ravi, including his branch and marks. The /api/students URL successfully returned the complete student information in JSON format. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to create and run an HTTP server using the Node.js http module.
- LO2: Ability to handle different request URLs and display student information dynamically.
- LO3: Ability to send HTML and JSON responses and handle individual student details using URL parameters.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
---------------------------	----------	------	----------

14	14		66
----	----	--	----

1. TITLE / AIM

Student Application using Express.js

To develop a Student Management application using Express.js with an appropriate folder structure, routes for the Home, Students and About pages, handling of GET and POST requests, custom middleware, static file handling and a template engine for displaying the student information.

2. OBJECTIVES

- To create an Express application with an appropriate folder structure.
- To create routes for the Home, Students and About pages.
- To handle GET and POST requests for adding and displaying student records.
- To create middleware that displays a message for every incoming request.
- To serve CSS, JavaScript and image files using static file handling. ● To display the student information using the EJS template engine.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: Node.js 20.x LTS, npm, Visual Studio Code, Google Chrome.
- Packages: express 4.x, ejs 3.x (installed through npm install express ejs). ● Languages: JavaScript (CommonJS), HTML, CSS.

4. THEORY / CONCEPT

Express.js is a minimal and flexible web application framework built on top of the http module of Node.js. It removes the repetitive work involved in manual routing, parsing of the request body and serving of static files, and provides a structured way of composing an application from middleware functions.

Routing in Express is defined through methods of the application object that correspond to the HTTP verbs, such as `app.get(path, handler)` and `app.post(path, handler)`. The handler receives the request object, the response object and the next function. The response is sent through `res.send()` for text or HTML, `res.json()` for a JSON payload, `res.render()` for a template and `res.redirect()` for a redirection. Route parameters are declared as `/student/:roll` and read from `req.params`, query string values are read from `req.query`, and submitted form data is read from `req.body`.

Middleware is a function that has access to the request and the response objects and to the next function in the chain. It may modify the request, terminate the response or call `next()` to pass control onward. Middleware is registered with `app.use()` and is executed in the order of

registration. Express provides the built-in middleware `express.static(folder)` for serving static assets, `express.urlencoded()` for parsing submitted form data and `express.json()` for parsing a JSON body.

A template engine generates HTML at run time by combining a template file with the data supplied by the application. EJS embeds JavaScript in the template, where the tag `<%= value %>` outputs an escaped value, `<%- value %>` outputs unescaped markup and `<% code %>` holds a control structure such as a loop. The engine is selected with `app.set("view engine", "ejs")`, and the templates are placed by convention in a folder named `views`.

A conventional Express project is organised with `app.js` at the root, the routes in a `routes` folder, the templates in a `views` folder and the static assets in a `public` folder, which keeps the routing logic separate from the presentation and simplifies maintenance.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Create an Express.js application with separate folders for routes, views and public files.
- Step 2: Configure EJS as the template engine and serve CSS, JavaScript and image files using `express.static()`.
- Step 3: Create middleware to display a message for every incoming request.
- Step 4: Define GET routes for Home, Students and About pages.
- Step 5: Create a POST route to accept and add new student details.
- Step 6: Use EJS templates to display the student records dynamically.

6. IMPLEMENTATION / SOURCE CODE

Folder structure

```
MSWD/  
├── exp14/  
│   ├── app.js  
│   ├── routes/  
│   │   └── studentRoutes.js  
│   ├── views/  
│   │   ├── index.ejs  
│   │   ├── students.ejs  
│   │   └── about.ejs  
│   └── public/  
│       ├── style.css  
│       └── script.js
```

app.js

```
const express = require("express");  
const path = require("path");  
  
const app = express();
```

```
const studentRoutes = require("./routes/studentRoutes");

app.set("view engine", "ejs");

app.use(express.urlencoded({ extended: true }));

app.use((req, res, next) => {
  console.log("Incoming Request:", req.method, req.url);
  next(); });

app.use(express.static(path.join(__dirname,
"public"))); app.use("/", studentRoutes);

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

```
routes/studentRoutes.js const
express = require("express");
const router = express.Router();
```

```
let students = [
  { name: "Ravi", branch: "CSE", marks: 85 },
  { name: "Sita", branch: "ECE", marks: 90 },
  { name: "Rahul", branch: "EEE", marks: 78 }
];
```

```
router.get("/", (req, res) => {
  res.render("index");
});
```

```
router.get("/students", (req, res) => {
  res.render("students", { students: students });
});
```

```
router.post("/students", (req, res) => {
  students.push({
    name: req.body.name,
    branch: req.body.branch,
    marks: req.body.marks
  });
});
```

```
    res.redirect("/students");  
  });
```

```
router.get("/about", (req, res) => {  
  res.render("about");  
}); module.exports =
```

```
router;
```

views/index.ejs

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Student Application</title>  
  <link rel="stylesheet" href="/style.css">  
</head>  
<body>  
  <h1>Welcome to Student Management Application</h1>  
  <a href="/students">Students</a>  
  <a href="/about">About</a>  
  
  <script src="/script.js"></script>  
</body>  
</html>
```

views/students.ejs

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Students</title>  
  <link rel="stylesheet" href="/style.css">  
</head>  
<body>  
<h1>Student Information</h1>  
  
<form method="POST" action="/students">  
  <input type="text" name="name" placeholder="Name" required>  
  <input type="text" name="branch" placeholder="Branch" required>  
  <input type="number" name="marks" placeholder="Marks" required>  
  <button type="submit">Add Student</button>
```

```
</form>
```

```
<h2>Student List</h2>
```

```
<% students.forEach(student => { %> <p>
    Name: <%= student.name %> |
    Branch: <%= student.branch %> |
    Marks: <%= student.marks %>
  </p>
<% }) %>
```

```
<a href="/">Home</a>
```

```
</body>
```

```
</html>
```

views/about.ejs

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>About</title>
```

```
  <link rel="stylesheet" href="/style.css">
```

```
</head>
```

```
<body>
```

```
  <h1>About Student Application</h1>
```

```
  <p>This application manages student information using Express.js and EJS.</p> <a
```

```
    href="/">Home</a>
```

```
</body>
```

```
</html>
```

public/style.css

```
body {
```

```
  font-family: Arial;
  margin: 30px; }
```

```
a { margin-right:
  15px;
}
```

```
input, button {  
  margin: 5px;  
  padding: 8px; }
```

public/script.js

```
console.log("Student Application loaded successfully.");
```

7. OBSERVATION / OUTPUT



Welcome to Student Management Application

[Students](#) [About](#)



Student Information

Name	Branch	Marks	Add Student
-----------------------------------	-------------------------------------	------------------------------------	--

Student List

Name: Ravi | Branch: CSE | Marks: 85

Name: Sita | Branch: ECE | Marks: 90

Name: Rahul | Branch: EEE | Marks: 78

[Home](#)



Student Information

Name	Branch	Marks	Add Student
-----------------------------------	-------------------------------------	------------------------------------	--

Student List

Name: Ravi | Branch: CSE | Marks: 85

Name: Sita | Branch: ECE | Marks: 90

Name: Rahul | Branch: EEE | Marks: 78

Name: Iris | Branch: CSE | Marks: 89

[Home](#)



About Student Application

This application manages student information using Express.js and EJS.

[Home](#)

8. RESULT / INFERENCE

The Student Application was successfully developed using Express.js with a proper folder structure. Routes for the Home, Students and About pages were implemented, and GET and POST requests were handled for displaying and adding student records. Custom middleware was used to display incoming request information, while static files were served using Express.js. The student details were dynamically displayed using the EJS template engine. Hence the objective of the experiment was achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to develop an Express.js application using routes, middleware and a structured folder organization.
- LO2: Ability to handle GET and POST requests and serve static resources such as CSS and JavaScript files.
- LO3: Ability to use EJS templates to dynamically display and update student information in a web application.

EXPERIMENT / ACTIVITY No.	WEEK No.	DATE	PAGE No.
15	15		74

1. TITLE / AIM

Student Database Application using MongoDB and Express.js

To develop a Student Management System using MongoDB that performs insertion, retrieval, updating and deletion of student records, applies filtering using comparison and logical operators, performs aggregation to obtain the average and the highest marks and the branch-wise count, and applies document validation and indexing.

2. OBJECTIVES

- To insert student records containing roll number, name, branch, year and marks.
- To retrieve all the records and the records of a particular branch.
- To update the marks or the branch of a student.
- To delete a student record using the roll number.
- To filter the records using comparison and logical operators.
- To perform aggregation for the average marks, the highest marks in each branch and the branch-wise count.
- To apply a validation rule to the student documents and to create an index on the roll number.

3. REQUIREMENTS / TOOLS USED

- Hardware: Personal computer with 4 GB RAM (minimum).
- Software: MongoDB 7.x Community Server, MongoDB Shell (mongosh), MongoDB Compass, Node.js 20.x LTS, Visual Studio Code.
- Packages: express 4.x, mongoose 8.x (installed through npm install express mongoose).
- Database: studentDB with the collection students.
- Languages: JavaScript (CommonJS), MongoDB Query Language.

4. THEORY / CONCEPT

MongoDB is a document-oriented NoSQL database in which the data is stored as BSON documents inside collections. A document is a set of field and value pairs similar to a JSON object, and unlike a row in a relational table it need not follow a fixed schema, which allows documents in the same collection to contain different fields.

The basic create, read, update and delete operations are performed through the methods insertOne() and insertMany(), find() and findOne(), updateOne(), updateMany() and replaceOne(), and deleteOne() and deleteMany(). The first argument of a read, update or delete

operation is a filter document that describes which records are to be selected; an empty filter selects every document in the collection.

A filter is expressed with query operators. The comparison operators are \$eq, \$ne, \$gt, \$gte, \$lt, \$lte, \$in and \$nin, and the logical operators are \$and, \$or, \$not and \$nor. The update operators include \$set to assign a value, \$inc to increase a numeric field, \$unset to remove a field and \$push to append to an array. A projection document supplied as the second argument of find() controls which fields are returned.

The aggregation framework processes documents through a pipeline of stages, in which the output of one stage becomes the input of the next. The commonly used stages are \$match for filtering, \$group for grouping with accumulators such as \$sum, \$avg, \$max and \$min, \$project for reshaping, \$sort for ordering and \$limit for restricting the number of results. Grouping is performed on the expression given in the _id field of the \$group stage.

Schema validation is applied to a collection through a \$jsonSchema validator, which specifies the required fields, their BSON types and the permitted ranges, so that a document violating the rule is rejected at the time of insertion. An index is a data structure that stores the values of a field in a sorted order and allows a query to locate the matching documents without scanning the whole collection; a unique index additionally prevents duplicate values, which makes it appropriate for the roll number.

Mongoose is an object document mapper that provides a schema-based interface to MongoDB from a Node.js application. A schema declares the structure and the validation rules, a model is compiled from the schema and exposes the query methods, and the connection is established through mongoose.connect() with the connection string of the server.

5. ALGORITHM / DESIGN / APPROACH

- Step 1: Connect Express.js with MongoDB and create the Student collection with the required fields.
- Step 2: Insert student records containing roll number, name, branch, year and marks.
- Step 3: Retrieve all student records and filter records based on the branch.
- Step 4: Update the marks or branch of a student using the roll number.
- Step 5: Delete a student record using the roll number.
- Step 6: Apply comparison and logical operators to filter student records.
- Step 7: Perform aggregation to calculate average marks, highest marks in each branch and branch-wise student count.
- Step 8: Apply document validation and create an index on the roll number field.

6. IMPLEMENTATION / SOURCE CODE

```
const express = require("express"); const {  
MongoClient } = require("mongodb");
```

```
const app = express();
app.use(express.json());

const url =
"mongodb://127.0.0.1:27017"; const
client = new MongoClient(url); let
students;

async function startServer() {
  await client.connect();

  const db = client.db("studentDB");
  students = db.collection("students");

  // Insert records await
  students.deleteMany({});

  await students.insertMany([
    { roll: 101, name: "Ravi", branch: "CSE", year: 2, marks: 85 },
    { roll: 102, name: "Sita", branch: "ECE", year: 3, marks: 90 },
    { roll: 103, name: "Rahul", branch: "CSE", year: 2, marks: 78 },
    { roll: 104, name: "Anu", branch: "EEE", year: 3, marks: 88 },
    { roll: 105, name: "Arun", branch: "ECE", year: 2, marks: 92 }
  ]);

  // Index on roll number
  await students.createIndex({ roll: 1 }, { unique: true });

  app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
  });
}

// Retrieve all students
app.get("/students", async (req, res) => {
  const data = await students.find().toArray();
  res.json(data);
});
```

```
// Retrieve students by branch
app.get("/students/branch/:branch", async (req, res) => {
  const data = await students
    .find({ branch: req.params.branch })
    .toArray();

  res.json(data);
});

// Update marks or branch
app.put("/students/:roll", async (req, res) => {
  await students.updateOne(
    { roll: Number(req.params.roll) },
    { $set: req.body }
  );

  res.json({ message: "Student updated successfully" });
});

// Delete student
app.delete("/students/:roll",
  async (req, res) => {
    await students.deleteOne({
      roll: Number(req.params.roll)
    });

    res.json({ message: "Student deleted successfully" });
  });

// Comparison and logical operators
app.get("/filter", async (req, res) => {
  const data = await students.find({
    $and: [
      { marks: { $gte: 80 } },
      { $or: [{ branch: "CSE" }, { branch: "ECE" }] }
    ]
  }).toArray();
  res.json(data);
});
```

```
// Aggregation app.get("/aggregate",
async (req, res) => {
  const average = await students.aggregate([
    {
      $group: {
        _id: null,
        averageMarks: { $avg: "$marks" }
      }
    }
  ]).toArray();

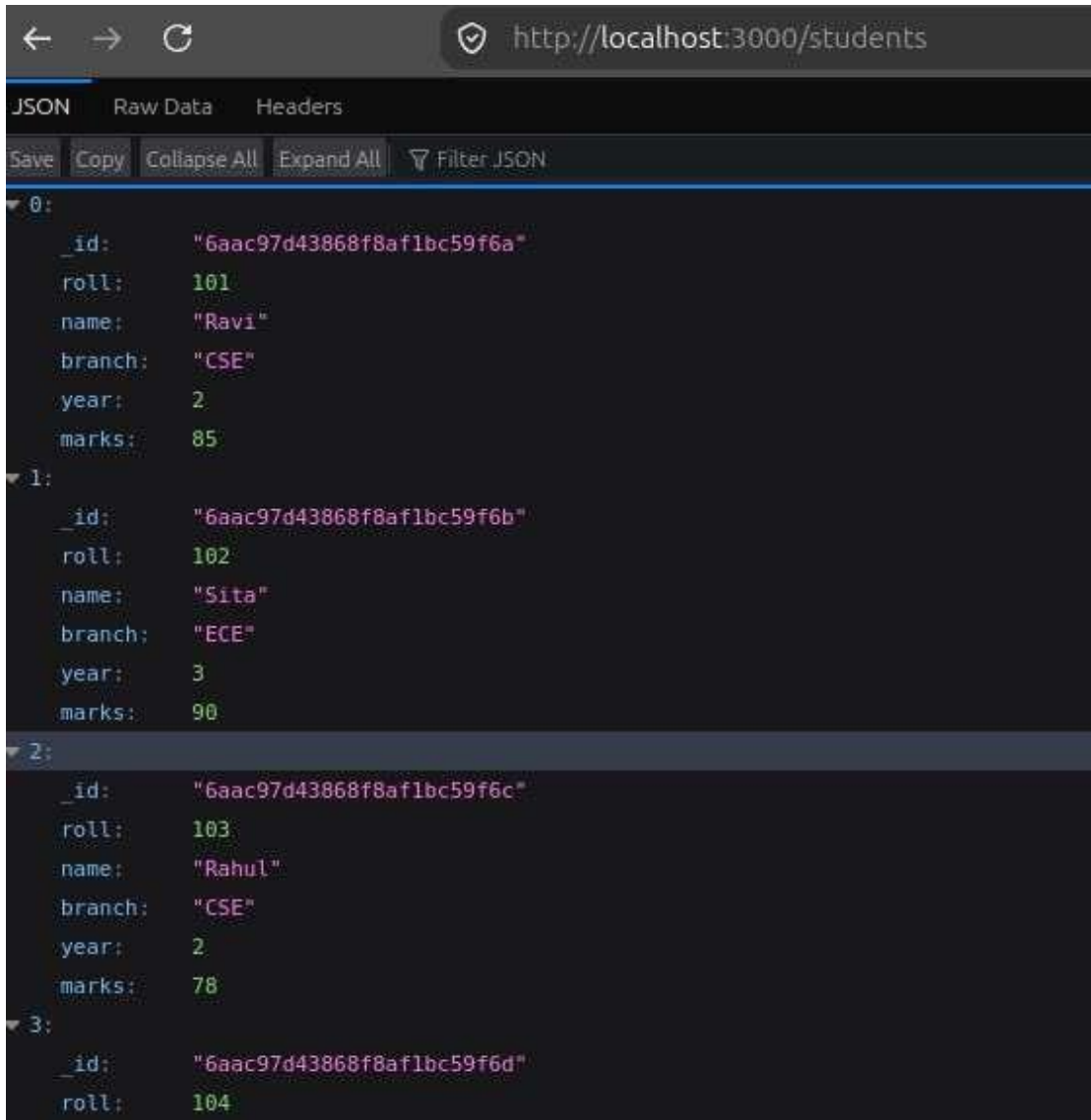
  const highest = await students.aggregate([
    {
      $group: { _id: "$branch", highestMarks: {
        $max: "$marks" }
      }
    }
  ]).toArray();

  const count = await students.aggregate([
    {
      $group: { _id: "$branch",
        studentCount: { $sum: 1 }
      }
    }
  ]).toArray();

  res.json({
    averageMarks: average,
    highestMarksByBranch: highest,
    branchWiseCount: count
  });
});

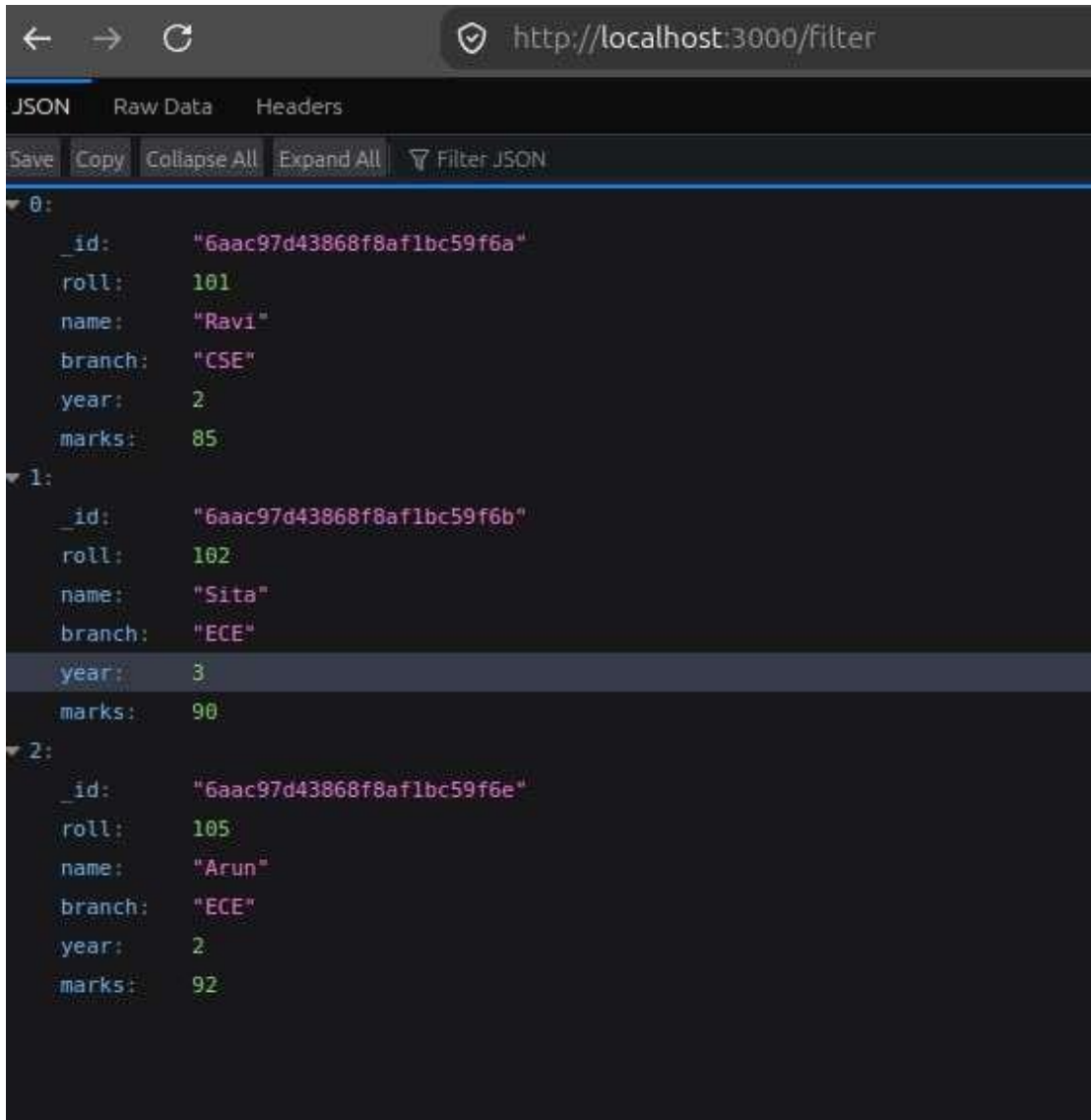
startServer().catch(console.error);
```

7. OBSERVATION / OUTPUT



```
← → ↻ http://localhost:3000/students
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
0:
  _id: "6aac97d43868f8af1bc59f6a"
  roll: 101
  name: "Ravi"
  branch: "CSE"
  year: 2
  marks: 85
1:
  _id: "6aac97d43868f8af1bc59f6b"
  roll: 102
  name: "Sita"
  branch: "ECE"
  year: 3
  marks: 90
2:
  _id: "6aac97d43868f8af1bc59f6c"
  roll: 103
  name: "Rahul"
  branch: "CSE"
  year: 2
  marks: 78
3:
  _id: "6aac97d43868f8af1bc59f6d"
  roll: 104
```

```
← → ↻ http://localhost:3000/students/branch/CSE
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON
0:
  _id: "6aac97d43868f8af1bc59f6a"
  roll: 101
  name: "Ravi"
  branch: "CSE"
  year: 2
  marks: 85
1:
  _id: "6aac97d43868f8af1bc59f6c"
  roll: 103
  name: "Rahul"
  branch: "CSE"
  year: 2
  marks: 78
```



The screenshot shows a web browser window with the address bar displaying `http://localhost:3000/filter`. The browser's developer tools are open, showing the JSON response. The JSON is an array of three objects, each representing a student. The second object, for Sita, is highlighted.

```
0: {
  _id: "6aac97d43868f8af1bc59f6a",
  roll: 101,
  name: "Ravi",
  branch: "CSE",
  year: 2,
  marks: 85,
}
1: {
  _id: "6aac97d43868f8af1bc59f6b",
  roll: 102,
  name: "Sita",
  branch: "ECE",
  year: 3,
  marks: 90,
}
2: {
  _id: "6aac97d43868f8af1bc59f6e",
  roll: 105,
  name: "Arun",
  branch: "ECE",
  year: 2,
  marks: 92,
}
```

```
{
  "averageMarks": {
    "0": {
      "_id": null,
      "averageMarks": 86.6
    }
  },
  "highestMarksByBranch": {
    "0": {
      "_id": "ECE",
      "highestMarks": 92
    },
    "1": {
      "_id": "CSE",
      "highestMarks": 85
    },
    "2": {
      "_id": "EEE",
      "highestMarks": 88
    }
  },
  "branchWiseCount": {
    "0": {
      "_id": "CSE",
      "studentCount": 2
    },
    "1": {
      "_id": "EEE",
      "studentCount": 1
    },
    "2": {
      "_id": "ECE",
      "studentCount": 2
    }
  }
}
```

ORGANIZATION: KAMBHAMPATI's Org... PROJECT: Project 0

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Cluster0

- admin
- local
- sample_mflix
- studentDB
 - students

students

Cluster0 > studentDB > students

Documents 5 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

EXPLAIN Reset Find Options

ADD DATA BULK EXPORT CODE

25 1-5 of 5

```
{
  "_id": ObjectId('6aac97d43868f8af1bc59f6a'),
  "roll": 101,
  "name": "Ravi",
  "branch": "CSE",
  "year": 2,
  "marks": 85
}
```

Alert System Status: All Good

©2020 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

8. RESULT / INFERENCE

The Student Database Application was successfully developed using Node.js, Express.js and MongoDB Atlas. Student records were inserted and retrieved successfully, including branch-wise retrieval. Update and delete operations were implemented using the student roll number, while comparison and logical operators were used for filtering. Aggregation operations successfully calculated the average marks as 86.6, the highest marks in each branch as ECE–92, CSE–85 and EEE–88, and the branch-wise student count as CSE–2, EEE–1 and ECE–2. An index was also created on the roll number.

During implementation, a connection error occurred because the local MongoDB server could not be started due to an incompatibility between the installed MongoDB version and the system kernel. The issue was resolved by using MongoDB Atlas instead of the local MongoDB server. The current IP address was added to the Atlas IP Access List, a database user was created, and the Atlas connection string was configured in the Node.js application. The application then connected successfully to Atlas and produced the required database and aggregation results. Hence, the objectives of the experiment were successfully achieved.

9. LEARNING OUTCOME

The following competencies were acquired through this experiment:

- LO1: Ability to connect an Express.js application with MongoDB Atlas and perform CRUD operations on student records.
- LO2: Ability to use MongoDB comparison and logical operators for filtering and aggregation operators for calculating average marks, highest marks and branch-wise student counts.
- LO3: Ability to create indexes, configure database connectivity and manage student information using MongoDB in a web application.